DEVOPS FINAL CAPSTONE

TaskFlow

One Git commit, from a laptop to a monitored Kubernetes cluster on AWS

Bhuvanesh M S · 24bcs10134

github.com/Bhuvanesh66/devops-heros · final-devops-project/

PROJECT OVERVIEW

What I built

The application

- A task tracker: title, description, priority, due date
- Tasks move **to do** → **in progress** → **done**
- Filters by status and live counters (per status, open high-priority)
- REST API with full CRUD, health, readiness and metrics

The real goal: the delivery chain

- Tests and security scans gate every commit
- Images tagged with the commit SHA in GHCR
- Terraform builds the AWS VPC and EKS
- Helm and Argo CD deploy to Kubernetes
- Prometheus and Grafana watch it run

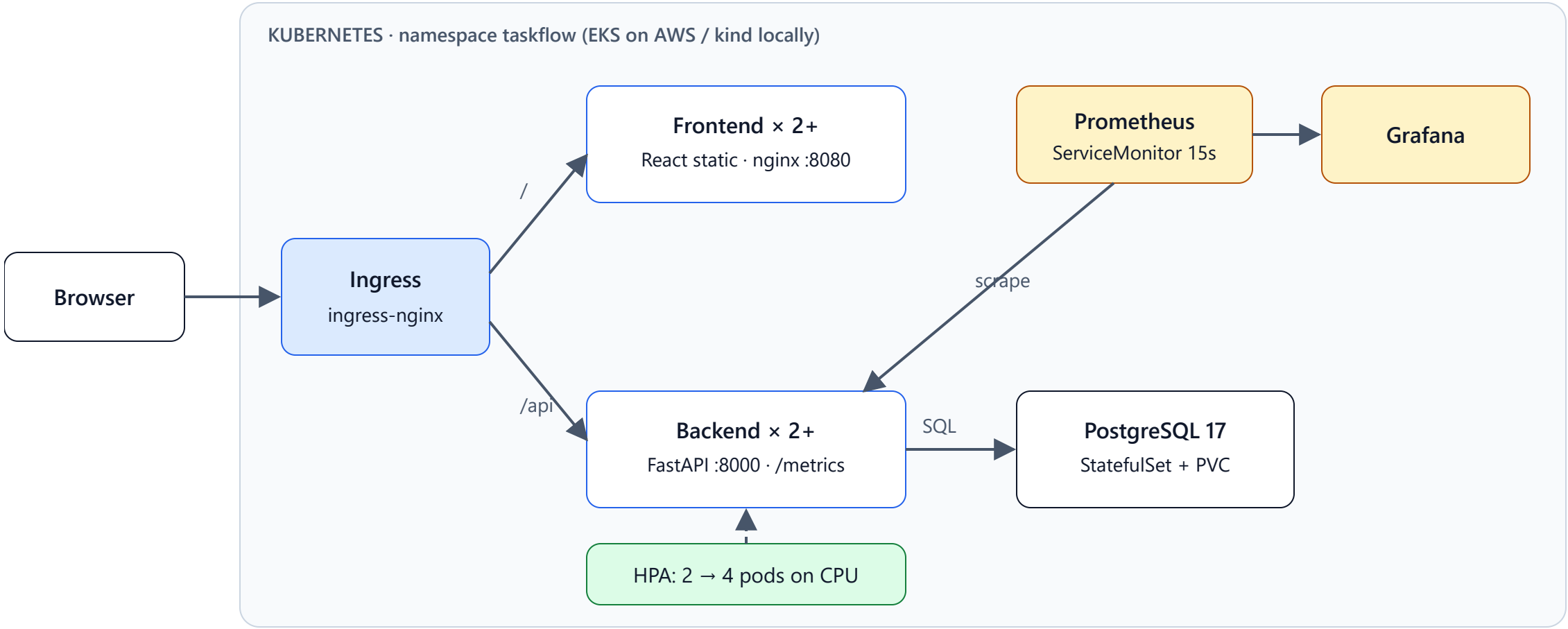
TECH STACK

Tools at every layer

LAYER	TECHNOLOGY
Frontend	React 19 + Vite 8, served by nginx-unprivileged
Backend	Python 3.12, FastAPI, SQLAlchemy 2, Alembic migrations
Database	PostgreSQL 17 (StatefulSet + PVC in Kubernetes)
Testing	pytest (18 tests, 90% coverage), ruff lint
Containers	Docker multi-stage builds, non-root, docker compose
CI/CD + Registry	GitHub Actions, GitHub Container Registry (GHCR)
DevSecOps	Semgrep, Bandit, pip-audit, npm audit, Gitleaks, Trivy
Infrastructure	Terraform → AWS VPC, EKS, EC2, S3 (ap-south-1)
Deploy / GitOps	Kubernetes, Helm, Ingress, HPA, Argo CD
Observability	Prometheus, Grafana, Alertmanager (kube-prometheus-stack)

ARCHITECTURE

How a request flows at runtime



CI/CD FLOW

GitHub Actions: every push to main

git push → Lint + pytest + frontend build → Validate IaC → SAST · SCA · Secret scan → Docker build + smoke test → Trivy image scan + SBOM → **Security gate** → Push to GHCR **:<sha>** → Helm deploy + helm test

Tests gate the build

A failing test stops the pipeline. No image is built or pushed.

SHA tags, not latest

Every image traces back to one exact commit, which makes rollback precise.

Scanned = shipped

The tar Trivy scanned is the one pushed. Image IDs are checked before push.

DEVSECOPS

Scanners report, one gate decides

CHECK	TOOL	RESULT
SAST	Semgrep (233 rules) + Bandit	0 findings
Dependencies (SCA)	pip-audit, npm audit, Trivy fs	0 vulnerabilities
Secrets	Gitleaks: files + git history	no leaks
Frontend image	Trivy	0 HIGH / CRITICAL
Backend image	Trivy	44 HIGH, 0 fixable

The 44 HIGH are all Debian 13 base packages (util-linux, perl…) with **no upstream fix yet**, e.g. CVE-2026-76642. The gate blocks any HIGH/CRITICAL **with a fix**. The app runs as non-root and never calls `mount`.

INFRASTRUCTURE AS CODE

Terraform on AWS (ap-south-1)

- **VPC** 10.20.0.0/16 with an Internet Gateway
- **2 public subnets** in ap-south-1a and 1b
- **EKS 1.34** cluster with a managed **node group**
- EBS CSI add-on so Postgres gets a volume
- EC2 Docker host (IMDSv2, encrypted disk) + S3 bucket
- IAM roles with only the AWS-managed policies needed

Workflow

`init → fmt → validate → plan → apply → destroy`

No secrets in Git

Credentials come from the AWS CLI. Only `terraform.tfvars.example` is committed.

Cost control

EKS is not free tier, so it's created for the demo and destroyed right after.

KUBERNETES + HELM

One chart, every environment

OBJECT	SETUP
Deployments	backend + frontend, 2+ replicas, rolling update
Services	ClusterIP :8000 and :80
Ingress	`/` → frontend, `/api` → backend
HPA	backend 2–4 on CPU
Probes	startup, liveness `/health`, readiness `/ready`
Config	ConfigMap + Secret, no default password
Database	StatefulSet + PVC

- `values-dev.yaml` / `values-prod.yaml`
- `helm upgrade --install` with the commit SHA
- `helm test` checks the live API
- `helm rollback` in seconds
- Checksum annotations roll pods when config changes

GITOPS

Argo CD: Git is the source of truth

Change image tag in values-dev.yaml → git push → Argo CD detects the change → Sync to cluster

- **Automated sync** with **prune** and **self-heal**: a Service deleted by hand came back in 2 seconds
- Rollback = `git revert`, with a full audit trail in Git history
- Fixed a permanent OutOfSync on the Postgres StatefulSet with `ignoreDifferences` on `volumeClaimTemplates`

OBSERVABILITY

Prometheus + Grafana

Metrics

`/metrics`: request rate, latency histograms, status codes, plus tasks created / completed / deleted

Scraping

kube-prometheus-stack. A ServiceMonitor scrapes every backend pod every 15 s, all targets **UP**

Dashboard

12 Grafana panels: traffic, 5xx ratio, p50/p95/p99 latency, CPU/memory, HPA replicas

Alert rules

Backend down · 5xx ratio > 5% · p95 latency > 0.5 s · pod restarts · HPA at max replicas

TROUBLESHOOTING

8 failures broken on purpose, then fixed

BROKEN	SYMPTOM
Image tag that doesn't exist	ImagePullBackOff
Wrong DB host in the ConfigMap	Init:CrashLoopBackOff
Service selector doesn't match the pods	No endpoints, 502/503
Wrong Service targetPort	Connection refused
Readiness probe on the wrong path	0/1 READY, rollout stuck
Secret key renamed	CreateContainerConfigError
No resource requests	HPA targets <unknown>
Frontend points at a missing Service	nginx CrashLoopBackOff

KEY LEARNINGS

What building it for real taught me

- **Readiness ≠ liveness.** If liveness checked the DB, one Postgres hiccup would restart every pod at once.
- **Only a real apply proves IaC.** IAM and S3 rejected brackets in a tag that EC2 accepted, and `validate` couldn't catch it.
- **Account limits matter.** An AWS Free-plan account refuses `t3.medium`, so the EKS node group hung until I switched to `m7i-flex.large`.
- **Scanners report, the gate decides.** A missing report counts as a failure, so the gate can't pass by accident.
- **GitOps and immutable fields.** API-server defaults in a StatefulSet kept Argo CD OutOfSync. The fix came from diffing the state, not from syncing again.

NEXT

Live demo

Code → push → pipeline → GHCR → Terraform + AWS → Argo CD → Kubernetes → Prometheus → Grafana

Thank you